
LUMIA: AN AI-POWERED WEB-BASED RESEARCH PAPER RETRIEVAL AND RECOMMENDATION SYSTEM

USER & TECHNICAL OPERATION MANUAL (Lumia Version 2.0)



ACADEMIC DEPARTMENT:

Department of Computer Studies

ACADEMIC YEAR:

2025-2026

PREPARED BY:

RYAN ANDREW A. REYES • MARIA MANELUZ A. ORANG • PRINCE ISIAH R.
BILLONES

System Operation & Technical Guide

Table of Contents

Document Section	Page
Introduction	2
1. System Core Concept	3
2. User Privilege Matrix	3
3. Getting Started	4
4. Features & Functionality (Student Portal)	5
5. Features & Functionality (Faculty & Admin)	6
6. Installation & Deployment Guide (Prerequisites & Backend)	8
7. Installation & Deployment Guide (Frontend & Launch)	9
8. Installation & Deployment Guide (PostgreSQL & Maintenance)	10
9. Technical Troubleshooting Guide	11
10. Technical Support, Authors & Glossary	12

Introduction

Welcome to the Lumia User & Technical Operation Manual. This manual provides comprehensive documentation for understanding, installing, operating, and maintaining the Lumia Research Retrieval System—an AI-powered, web-based capstone and thesis repository utilizing advanced BERT-NLP semantic search and IMRAD-aware structure extraction.

This system replaces traditional keyword matching database lookups with deep natural language queries, allowing students, faculty, and researchers to locate academic documents based on contextual meaning rather than matching words. This document guides students through query techniques, faculty through document ingestion/OCR, admins through approvals, and IT staff through technical installations, environment configurations, and deployment procedures.

1. System Core Concept

LUMIA is an advanced, AI-powered web-based research paper repository designed to catalog and retrieve institutional capstones and undergraduate theses. Traditional search systems rely on database keyword matching, which limits searches to exact words. Lumia resolves this by employing semantic vector search using BERT-NLP (via the Sentence-Transformers model: multi-qa-MiniLM-L6-cos-v1).

Additionally, the system utilizes a unique IMRAD-Aware Retrieval strategy. When research PDFs are uploaded, the backend runs Optical Character Recognition (OCR) and splits the extracted text into structural sections: Introduction, Methods, Results, and Discussion (IMRAD). These sections are embedded as independent high-dimensional vectors in a local or cloud Qdrant Vector Database. This enables users to perform targeted semantic searches, such as querying specifically for a methodology or statistical technique.

2. User Privilege Matrix

Access to the system features is guarded by a role-based authentication model. Users are mapped to specific groups depending on academic roles.

System Capability	Student (User)	Faculty Member	Administrator
Search & Academic Filtering	Yes	Yes	Yes
Read Metadata & Abstract	Yes	Yes	Yes
Upload Research Documents	Yes (Requires Approval)	Yes	Yes
Approve Pending Uploads	No	No	Yes
User Accounts Moderation	No	No	Yes
System Logs & Audit Control	No	No	Yes

3. Getting Started

3.1 Registration & Authentication

Users access the system through a secure registration and login process. To create a student account, navigate to the Register portal (reg_win.vue), enter your full name, student email, preferred username, and password. Faculty and Administrator accounts are created directly by administrators or require role upgrading. Access to protected areas is gated by JWT tokens; attempts to access these pages without authenticating will redirect to the login module (auth_win.vue).

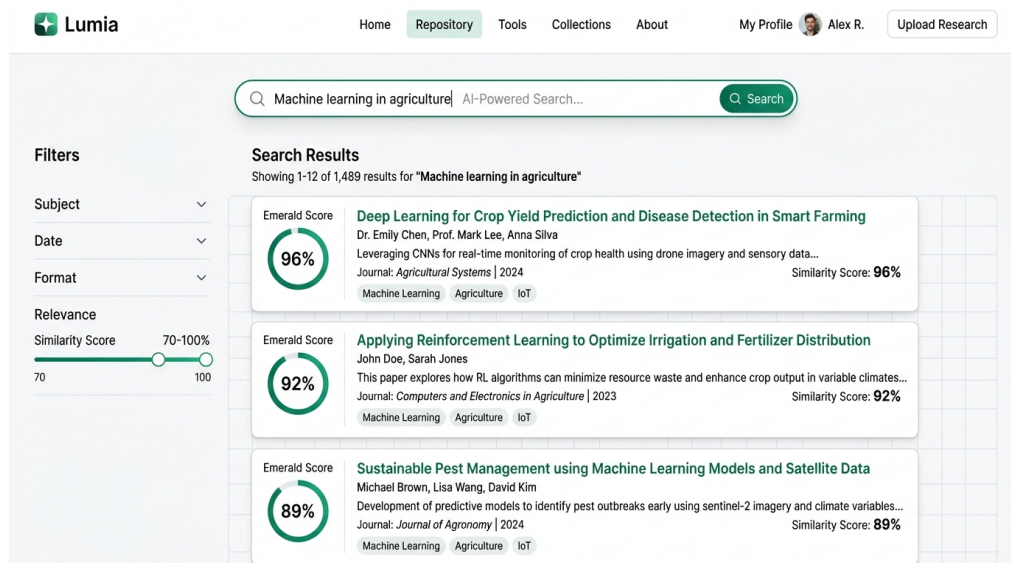
3.2 Basic System Navigation

Lumia features a theme-aware interface supporting dark and light modes. Theme preference is automatically synced with user profiles. Key navigation components include the central search dashboard, the document upload wizard, and the admin control panel. The layout is optimized to adjust dynamically between mobile and desktop viewports, ensuring usability across multiple device profiles.

4. Features & Functionality

4.1 Student Portal & Search Interface

The primary discovery mechanism is the Explore Portal (explore_win.vue), which provides hybrid search capabilities. Relational SQL matching executes concurrently with Qdrant vector search to compute overall cosine similarity scores. This ensures accurate retrieval based on contextual meaning.



- Target Section Dropdown: Focuses semantic query matching to specific structural paper sections such as Introduction, Methodology, Results, or Discussion.
- Academic Filter Controls: Filters results by Department (BSCS, BSIT, BSCpE), Project Type (Capstone vs Thesis), or Publication Year ranges.

4.2 Document Detail View & Citations

Selecting a search result loads the Paper Detail View (detail_win.vue). This view shows extracted metadata, structured section contents, figures, and citation listings. The recommendation engine computes 'Narrow Down' recommendations by comparing the methods vector of the active document with overall index entries. Dynamic citation generators export reference strings in APA 6th, 7th, and in-text citation styles.

4.3 Student Research Paper Submission

Students can submit their completed capstones and theses through the student upload portal (up_win.vue). Submissions are added as pending records and will not be vectorized or display in search indexes until an administrator reviews and approves the record.

4.4 Faculty Guided 3-Step Ingestion Flow

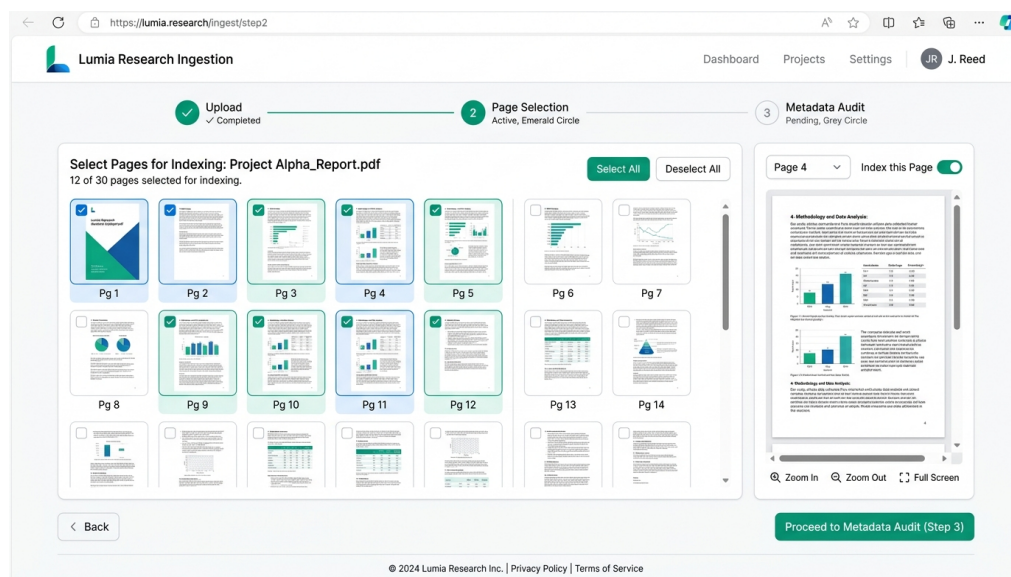
Faculty members ingest and index documents via a structured 3-step wizard in the upload portal:

Step 1: Document Upload & Ingestion Strategy Selection

The user uploads a PDF file and selects either Smart Extract (OCR) to auto-parse metadata and extract sections using PyTesseract, or Manual Input to input document details directly without OCR processing.

Step 2: Thumbnail Review & Page Indexing Selection

A grid of page thumbnails displays side-by-side with OCR-extracted text. Users click thumbnails to zoom and deselect non-academic pages (cover pages, signature forms, references) to prevent noise from contaminating vector databases.



Step 3: Metadata Auditing & Vector Indexing

The user reviews extracted metadata. Missing or incorrect fields are updated manually. Authors must be registered using the | pipe delimiter to format names correctly in individual entry boxes. Clicking Submit and Index saves metadata to SQL and indexes section vectors in Qdrant.

□ *TIP: Page Selection Accuracy*

Deselecting non-academic content pages (e.g., acknowledgement lists, reference sheets) in Step 2 directly improves the semantic retrieval accuracy of the system, since only pure technical information will be vectorized.

4.5 Administrator Management Console

Administrators access the Management Deck (`manage_win.vue`) to oversee platform operations. This includes viewing system dashboard analytics, managing user accounts, upgrading roles to Faculty or Admin, and approving or deleting pending student documents. The audit log screen monitors relational events (uploads, edits, deletions) recorded directly from the database.

5. Installation & Deployment Guide

This section describes system deployment procedures, prerequisite packages, environment variables, and operational commands for running the Lumia platform.

5.1 System Requirements & Prerequisites

Before setting up the repositories, install the following external system-level binaries:

1. Tesseract OCR: Install UB-Mannheim Tesseract OCR installer. Add the installation folder to the system's Environment PATH variable.
2. Poppler Library: Extract Poppler Windows binaries. Add the /bin subdirectory to the system's Environment PATH variable to enable PDF rendering.
3. Node.js: Install LTS version 20.0 or higher.
4. Python: Install Python version 3.10 or higher.
5. Docker: Install Docker Desktop to run a production-ready instance of Qdrant Vector Database.

5.2 Vector Database Setup

Qdrant stores the 6-vector IMRAD schema. For local testing, it runs in file-storage mode. For production, run the Docker image:

```
docker run -d -p 6333:6333 -p 6334:6334 \
  -v qdrant_storage:/qdrant/storage \
  qdrant/qdrant
```

5.3 Backend Environment Configuration (FastAPI)

Create a .env file in the backend root directory lumia_retrieval_bert:

```
# Relational database (Default SQLite for dev, PostgreSQL for production)
DATABASE_URL=sqlite:///./thesis.db
# Vector database configuration
QDRANT_URL=http://localhost:6333
# Toggle Sample Document Ingestion
ENABLE_SAMPLE_DOCS=false
```

5.4 Frontend Environment Configuration (Vue 3)

Create a `.env` file in the frontend root directory `lumia_frontend_sample` to link with backend API:

```
# Frontend API URL configuration (Point to local or deployed backend API)
VITE_API_BASE_URL=http://localhost:8000/api/v1
```

5.5 Service Initialization & Launch Commands

To start the system, open PowerShell terminal windows in the respective workspaces:

```
# BACKEND COMMANDS (FastAPI)
cd lumia_retrieval_bert
./setup_local.ps1          # Initialize venv and packages
venv/Scripts/activate      # Activate virtual env
python migrate.py          # Run schema migrations
python create_admin.py     # Seed default admin accounts
uvicorn app.main:app --reload # Start web backend API

# FRONTEND COMMANDS (Vue/Vite)
cd lumia_frontend_sample
npm install                # Install Node modules
npm run dev                # Run dev compilation server
```

5.6 Production Deployment & PostgreSQL Migration

To support multi-user network deployment and prevent SQLite file locking, migrating to a central PostgreSQL database is required. Provision a PostgreSQL instance (e.g., on Railway, Supabase, or self-hosted) and modify the DATABASE_URL to: postgresql://username:password@host:port/database. Run the schema migrations script (migrate.py) on deployment to recreate all table schemas on PostgreSQL.

5.7 Maintenance and Reset Operations

If you need to clear the vector database collections or purge uploaded PDF files during system initialization, run the reset utility script in the backend directory:

```
python reset_storage.py
```

This script purges all physical files in the uploads/ directory, resets SQLite/PostgreSQL table schemas, and completely deletes/recreates the vector collections in Qdrant.

6. Technical Troubleshooting Guide

For system administrators deploying or evaluating the system, use the reference matrix below to resolve common environmental and service configuration errors:

Error / Symptom	Probable Cause	Actionable Resolution
PDFInfoNotInstalledError or OCR fails to render pages.	The poppler PDF utility binaries are not installed or not configured in system environment variables.	Download Poppler Windows binaries, extract files, and append the absolute path of the /bin folder to the system's PATH. Restart terminal sessions.
TesseractNotFound or text extraction fails.	The tesseract-ocr executable is missing or not configured in system PATH.	Install the Tesseract OCR application. Add its installation folder (typically C:\Program Files\Tesseract-OCR) to the system environment PATH. Restart terminals.
ConnectError [Errno 111] or port 6333 Refused.	Qdrant service is inactive or network port binding is blocked.	Ensure the Qdrant Docker container is running (docker start qdrant) or double-check local folder permissions if running Qdrant in file-storage mode.
OperationalError or (no such table / columns).	Relational database schemas are not initialized or out of sync.	Run the SQLAlchemy schema migration command in backend: python migrate.py. This creates SQLite file structure or PostgreSQL tables.
CORS Errors / API Failures on frontend console.	The backend FastAPI host differs from the VITE_API_BASE_URL environment setup.	Verify FastAPI is active. Align frontend `.env` config (e.g., VITE_API_BASE_URL=http://localhost:8000/api/v1) with backend settings. Restart Vite dev server.

7. Technical Support & Authors

Proponent Names	Academic Department	Project Roles
Ryan Andrew A. Reyes	Computer Studies Department	BERT-NLP Integration & Database Admin
Maria Maneluz A. Orang	Computer Studies Department	Frontend Development (Vue 3/Vite) & UI
Prince Isiah R. Billones	Computer Studies Department	OCR Parsing Pipeline & Document Indexing

8. Glossary & Appendices

IMRAD: Introduction, Methodology, Results, and Discussion. A standard formatting structure for scientific research papers, capstones, and theses.

BERT-NLP: Bidirectional Encoder Representations from Transformers. A transformer-based machine learning technique for natural language processing pre-training developed by Google, used here to build semantic embeddings of paper sections.

Qdrant: A production-ready vector database and vector similarity search engine, used here to index and search high-dimensional IMRAD section vectors.

JWT: JSON Web Token. An open standard for securely transmitting information between parties as a JSON object, used for user authentication.

OCR: Optical Character Recognition. The electronic conversion of images of typed, handwritten or printed text into machine-encoded text, using Tesseract and Poppler here.

Vector Embeddings: Numerical representations of text semantics in a high-dimensional vector space, allowing mathematical calculation of semantic similarity.